AI ENGINEERING ROADMAP · STUDY NOTES

Chapter 04

Prompts

How to design, structure, and reuse the inputs you send to an LLM — from static text boxes to fully dynamic, template-driven, multi-turn conversations.

WHAT THIS CHAPTER COVERS

- What a prompt actually is
- Static prompts vs. dynamic prompts
- Building a research-tool UI with Streamlit
- The PromptTemplate class & why it beats f-strings
- Saving & reusing templates as JSON
- Building a context-aware command-line chatbot
- SystemMessage, HumanMessage & AIMessage
- ChatPromptTemplate & MessagesPlaceholder

Prompts

RECAP — CHAPTER 3

Chapter 3 covered the Model Component end to end: LLMs vs. Chat Models, closed-source providers (OpenAI, Anthropic, Gemini), open source models via Hugging Face, and embedding models. This chapter covers the second core LangChain component: **Prompts** — how the messages sent to a model are designed, made reusable, and kept consistent across users.

1. What Are Prompts?

Definition

A **prompt** is simply the message sent to an LLM. Prompts are not a new concept introduced in this chapter — every `.invoke()` call made with a Chat Model in earlier chapters already sent a prompt; it was just never explicitly called that.

Why Prompts Matter

An LLM's output is extremely sensitive to its input. Even a small change in wording can meaningfully change the response. Because of this sensitivity, a large set of techniques exists purely for designing effective prompts — an entire job role, *Prompt Engineering*, has emerged around this skill.

Prompt Modalities

Prompts are not limited to text. A model can also be prompted with an image, an audio clip, or a video, and asked questions about that content (multimodal prompting). This chapter, however, focuses entirely on **text-based prompts**, since that is what the overwhelming majority of real-world applications use today.

IMPORTANT

Because LLM output is highly sensitive to prompt wording, the biggest practical risk in an LLM application is inconsistency — different phrasing producing very different quality of answers. Everything in this chapter builds toward solving that consistency problem.

↻ Quick Revision — What Are Prompts?

- A prompt is simply the message sent to an LLM — nothing new, just a formal name for it.
- LLM output is highly sensitive to how a prompt is worded.
- Prompt Engineering is the discipline of designing effective prompts.
- Prompts can be multimodal (image, audio, video), but this chapter focuses on text-based prompts.

2. Static Prompts vs. Dynamic Prompts

The Problem: Where Should Prompts Come From?

In every example so far, the developer wrote the prompt directly inside `.invoke()` in the source code. In a real application this is backwards: the **user** of the application should supply the prompt (or the values that go into it), and the developer's code should take that input and forward it to the LLM.

Example Scenario: A Research Paper Summarizer

Consider a tool where a user pastes in the name of a research paper and asks for a summary. The most direct implementation gives the user a single free-text box and sends whatever they type straight to the LLM.

```
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from dotenv import load_dotenv
import streamlit as st

load_dotenv()

llm = HuggingFaceEndpoint(
    repo_id="Qwen/Qwen2.5-7B-Instruct",
)
model = ChatHuggingFace(llm=llm)

st.header("Research Tool")

# the user types the ENTIRE prompt themselves — this is a static prompt
user_input = st.text_input("Enter your prompt:")

if st.button("Summarize"):
    result = model.invoke(user_input)
    st.write(result.content)
```

Definition: Static Prompt

A **static prompt** is one where the user types out the complete, free-form prompt text themselves, with no fixed structure enforced by the application.

Why Static Prompts Are Risky

Handing the entire prompt to the user hands them a lot of control over the LLM's output — and since output is so sensitive to wording, this quickly becomes a liability:

- A user may misspell a paper's title, causing the LLM to hallucinate or answer incorrectly.
- A user may accidentally (or inconsistently) change the requested style or length, breaking the application's promised experience.
- There is no way to guarantee every user gets a consistent, high-quality result.

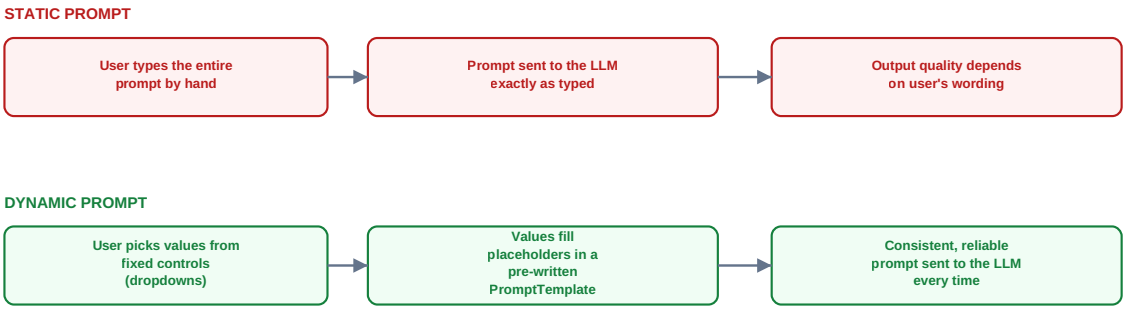

Figure 4.1 — Static prompts hand full control (and full risk) to the user; dynamic prompts constrain input to guarantee a consistent, reliable prompt.

Definition: Dynamic Prompt

A **dynamic prompt** is a pre-written template with placeholders, where the application collects only a few constrained values from the user (e.g. via dropdowns) and fills those placeholders in at runtime — producing a consistent, well-structured final prompt every time.

> **COMMON MISTAKE**
>
> Giving users a single open text box for a task that needs precise, structured input (like a paper title, an explanation style, and a desired length). This maximizes the chance of typos and inconsistent results reaching the LLM.

> ↻ **Quick Revision — Static vs. Dynamic Prompts**
>
> • Static prompt: the user types the full prompt text freely — flexible but risky.
>
> • Dynamic prompt: fixed template with placeholders filled from constrained user input (e.g. dropdowns).
>
> • Static prompts are rarely used in production because output quality becomes unpredictable.
>
> • Dynamic prompts trade a little user freedom for consistent, dependable application behavior.

3. Building a Dynamic Prompt with PromptTemplate

Step 1: Redesign the UI Around Fixed Inputs

Instead of one free-text box, the research tool now asks for three constrained inputs via dropdowns: which paper to summarize, the explanation style, and the explanation length.

```
paper_input = st.selectbox("Select Research Paper Name", [
    "Attention Is All You Need",
    "BERT: Pre-training of Deep Bidirectional Transformers",
    "GPT-3: Language Models are Few-Shot Learners",
    "Diffusion Models Beat GANs on Image Synthesis"
])

style_input = st.selectbox("Select Explanation Style", [
    "Beginner-Friendly", "Technical", "Code-Oriented", "Mathematical"
])

length_input = st.selectbox("Select Explanation Length", [
    "Short (1-2 paragraphs)", "Medium (3-5 paragraphs)", "Long (detailed explanation)"
])
```

Step 2: Design the PromptTemplate

The `PromptTemplate` class from `langchain_core.prompts` defines the fixed wording of the prompt once, with named placeholders (`{paper_input}`, `{style_input}`, `{length_input}`) that get filled in at runtime.

```
from langchain_core.prompts import PromptTemplate

template = PromptTemplate(
    template="""Please summarize the research paper titled "{paper_input}" with the
following specifications:
    Explanation Style: {style_input}
    Explanation Length: {length_input}
    1. Mathematical Details:
    - Include relevant mathematical equations if present in the paper.
    - Explain the mathematical concepts using simple, intuitive code snippets where
applicable.
    2. Analogies:
    - Use relatable analogies to simplify complex ideas.
    If certain information is not available in the paper, respond with: "Insufficient
information available" instead of guessing.
    Ensure the summary is clear, accurate, and aligned with the provided style and
length.""",
    input_variables=["paper_input", "style_input", "length_input"],
    validate_template=True
)

# save the template to disk so it can be reused elsewhere
template.save("template.json")
```

Step 3: Load the Template and Build a Chain

In the actual Streamlit app, the saved template is loaded back with `load_prompt()`, then combined with the model into a single **chain** using the pipe (`|`) operator — so filling the template and calling the model happen in one `.invoke()` instead of two.

```python
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from langchain_core.prompts import PromptTemplate, load_prompt
from dotenv import load_dotenv
import streamlit as st

load_dotenv()

llm = HuggingFaceEndpoint(repo_id="Qwen/Qwen2.5-7B-Instruct")
model = ChatHuggingFace(llm=llm)

st.header("Research Tool")

paper_input = st.selectbox("Select Research Paper Name", ["..."])
style_input = st.selectbox("Select Explanation Style", ["..."])
length_input = st.selectbox("Select Explanation Length", ["..."])

# load the previously saved template — fill the placeholders with user input
template = load_prompt("template.json")

if st.button("Summarize"):
    # the pipe operator chains the template directly into the model
    chain = template | model
    result = chain.invoke({
        "paper_input": paper_input,
        "style_input": style_input,
        "length_input": length_input
    })
    st.write(result.content)
```

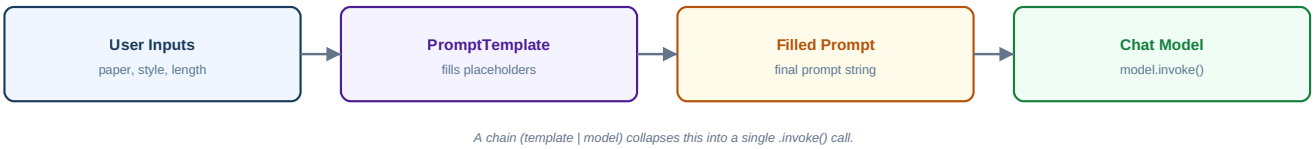

Figure 4.2 — A chain built with the pipe operator (template | model) fills the placeholders and calls the model in a single .invoke().

BEST PRACTICE

Design the full prompt wording once as a `PromptTemplate`, save it with `.save("template.json")`, and load it wherever it's needed with `load_prompt()` — this keeps the prompt's wording centralized and consistent across the entire application.

↻ Quick Revision — PromptTemplate

- PromptTemplate defines a fixed prompt string with named {placeholders}.
- input_variables lists every placeholder name that must be supplied.
- validate_template=True enables automatic validation of the placeholders.
- A saved template (.save()) can be reloaded anywhere with load_prompt().
- chain = template | model lets you fill the template and call the model in a single .invoke().

4. Why PromptTemplate Instead of f-strings?

The Natural Question

Since a `PromptTemplate` is just a big string with placeholders filled in at runtime, a Python f-string could technically do the same job. So why introduce a dedicated class at all? There are three concrete reasons.

1. BUILT-IN VALIDATION

With `validate_template=True`, LangChain automatically checks that every placeholder used inside the template string also appears in `input_variables` — and vice versa. If a placeholder is missing from the list, or an extra one is listed that isn't used in the template, the code raises an error immediately at development time rather than failing unpredictably in production.

2. REUSABILITY

A large template hard-coded inline makes application code bulky, and if the same template is needed on multiple pages, it must either be duplicated or imported. Saving a template as a standalone `.json` file (as shown in Section 3) makes it a portable, reusable asset — any file that needs it simply calls `load_prompt("template.json")`.

3. ECOSYSTEM INTEGRATION

Because a `PromptTemplate` is a proper LangChain object (not just a string), it plugs directly into other LangChain constructs — most notably **chains**, via the pipe operator (`template | model`). An f-string cannot be piped into a chain this way.

> **INTERVIEW TIP**
>
> If asked why LangChain has a PromptTemplate class when f-strings do the same string-filling job, give all three reasons: automatic validation of placeholders, reusability via save/load as JSON, and native compatibility with chains and the rest of the LangChain ecosystem.

↻ Quick Revision — PromptTemplate vs. f-strings

- f-strings can technically fill placeholders too, but lack built-in safety checks.
- PromptTemplate validates that placeholders in the template and input_variables match exactly.
- PromptTemplate can be saved and reloaded as JSON, making it reusable across files and pages.
- PromptTemplate integrates natively with chains via the pipe (|) operator; a plain f-string cannot.

5. Building a Command-Line Chatbot

Definition

A minimal chatbot runs an infinite loop: it repeatedly asks the user for input, sends that input to the LLM, prints the reply, and continues until the user types `"exit"`.

First Version (No Memory)

```
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from dotenv import load_dotenv

load_dotenv()

llm = HuggingFaceEndpoint(repo_id="Qwen/Qwen2.5-7B-Instruct")
model = ChatHuggingFace(llm=llm)

while True:
    user_input = input("You: ")
    if user_input == "exit":
        break
    result = model.invoke(user_input)  # a single static prompt, no history
    print("AI: ", result.content)
```

The Problem: No Context

Because each call to `model.invoke()` sends only the latest message, the model has no memory of anything said earlier in the conversation. For example, asking *"Which is greater, 2 or 0?"* then following up with *"Now multiply the bigger number by 10"* fails, because the model no longer knows which number was "the bigger number."

COMMON MISTAKE

Calling `model.invoke()` with only the newest user message in a multi-turn chatbot. Without the full conversation history attached, the model cannot resolve references to anything said earlier.

The Fix: Maintain a Chat History

The corrected version accumulates every message — both from the user and from the model — into a running list, and sends that entire list to `model.invoke()` on every turn.

```
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from dotenv import load_dotenv

load_dotenv()

llm = HuggingFaceEndpoint(repo_id="Qwen/Qwen2.5-7B-Instruct")
model = ChatHuggingFace(llm=llm)

chat_history = [
    SystemMessage(content="You are a helpful AI assistant.")
]

while True:
    user_input = input("You: ")
    chat_history.append(HumanMessage(content=user_input))
    if user_input == "exit":
        break
    result = model.invoke(chat_history)  # send the ENTIRE history, not just the latest
message
    chat_history.append(AIMessage(content=result.content))
    print("AI: ", result.content)

print(chat_history)
```

With the full history attached, the model can now correctly resolve the follow-up question, since "the bigger number" can be traced back through the conversation.

REAL WORLD INSIGHT

The `.invoke()` method is flexible enough to accept either a single message or a list of messages. Sending a list is what enables genuine multi-turn conversation.

↻ Quick Revision — Command-Line Chatbot

- A basic chatbot loop repeatedly reads user input and calls model.invoke() until 'exit'.
- Sending only the latest message means the model has no memory of earlier turns.
- Maintaining a running chat_history list and sending the entire list fixes this.
- model.invoke() accepts either a single message or a list of messages.

6. Messages in LangChain

The Remaining Problem

Storing every message as plain strings in a list works, but loses one critical piece of information: **who said it**. As a conversation grows, the model needs to know which lines were said by the user and which were generated by itself, or context tracking breaks down.

Definition

LangChain solves this with exactly three message types, each labeling who a piece of content came from:

Message Type	Represents	Example
SystemMessage	A system-level instruction sent once, at the start of a conversation, to set behavior or persona	`"You are a helpful assistant."`
HumanMessage	A message sent by the user to the LLM	`"Tell me about LangChain."`
AIMessage	A message returned by the LLM	`"LangChain is a framework for..."`

Example

```
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from dotenv import load_dotenv

load_dotenv()

llm = HuggingFaceEndpoint(repo_id="Qwen/Qwen2.5-7B-Instruct")
model = ChatHuggingFace(llm=llm)

messages = [
    SystemMessage(content="You are a helpful assistant."),
    HumanMessage(content="Tell me about LangChain."),
]

result = model.invoke(messages)

# label the model's reply and add it back to the running history
messages.append(AIMessage(content=result.content))

print(messages)
```

The printed list now clearly shows each entry's role (system, human, or AI) alongside its content and any provider-specific metadata — so no matter how long the conversation grows, the model can always tell who said what.

IMPORTANT

Every serious LangChain chatbot should build its chat history out of SystemMessage, HumanMessage, and AIMessage objects — not plain strings — so that speaker identity is preserved as the conversation grows.

↻ Quick Revision — Messages

- LangChain has exactly three message types: SystemMessage, HumanMessage, AIMessage.
- SystemMessage sets behavior/persona once, at the start of a conversation.
- HumanMessage represents user input; AIMessage represents the model's reply.
- Building chat history from these typed objects (instead of raw strings) preserves who said what.

7. ChatPromptTemplate for Dynamic Multi-Turn Messages

Definition

`ChatPromptTemplate` is the multi-message equivalent of `PromptTemplate`: it lets you build a *list* of messages where one or more of them contain dynamic placeholders, filled in at runtime.

Why It Exists

Single-message prompts built with `PromptTemplate` only cover one dynamic message at a time. But a real conversation often needs several messages to be dynamic together — for example, a system message whose persona/domain isn't fixed yet, paired with a human message whose topic isn't fixed yet. `ChatPromptTemplate` handles filling placeholders across an entire list of messages in one step.

Example

```
from langchain_core.prompts import ChatPromptTemplate

chat_template = ChatPromptTemplate([
    ("system", "You are a helpful {domain} expert"),
    ("human", "Explain in simple terms, what is {topic}")
])

prompt = chat_template.invoke({"domain": "cricket", "topic": "Dusra"})

print(prompt)
```

Running this fills both placeholders correctly, producing a system message ("You are a helpful cricket expert") and a human message ("Explain in simple terms, what is Dusra").

COMMON MISTAKE

Constructing messages by passing `SystemMessage(content="...{domain}...")` objects directly into `ChatPromptTemplate` and expecting the placeholders to be filled. In practice, this does not reliably fill the placeholders. Instead, pass each message as a `(role, template_string)` tuple — e.g. `("system", "You are a helpful {domain} expert")` — which is the pattern shown in LangChain's current documentation and is the reliable way to build a ChatPromptTemplate.

↻ Quick Revision — ChatPromptTemplate

- ChatPromptTemplate is the multi-message version of PromptTemplate.
- It fills placeholders across an entire list of role-tagged messages at once.
- Build it from (role, template_string) tuples — e.g. ('system', '...'), ('human', '...').
- Use ChatPromptTemplate whenever multiple messages in a conversation need to be dynamic together.

8. MessagesPlaceholder: Injecting Chat History

Motivating Scenario

Consider a customer support chatbot. A customer asks for a refund on order #12345; the bot replies that the refund has been initiated. The conversation ends there and is saved. Two days later, the same customer returns and asks, *"Where is my refund?"* — a brand new session. To answer correctly, the bot must first reload the entire prior conversation as context before it can make sense of this new, vague question.

Definition

A **MessagesPlaceholder** is a special placeholder used inside a `ChatPromptTemplate` to dynamically insert an entire list of past messages (a chat history) at a specific position, at run time.

How It Works

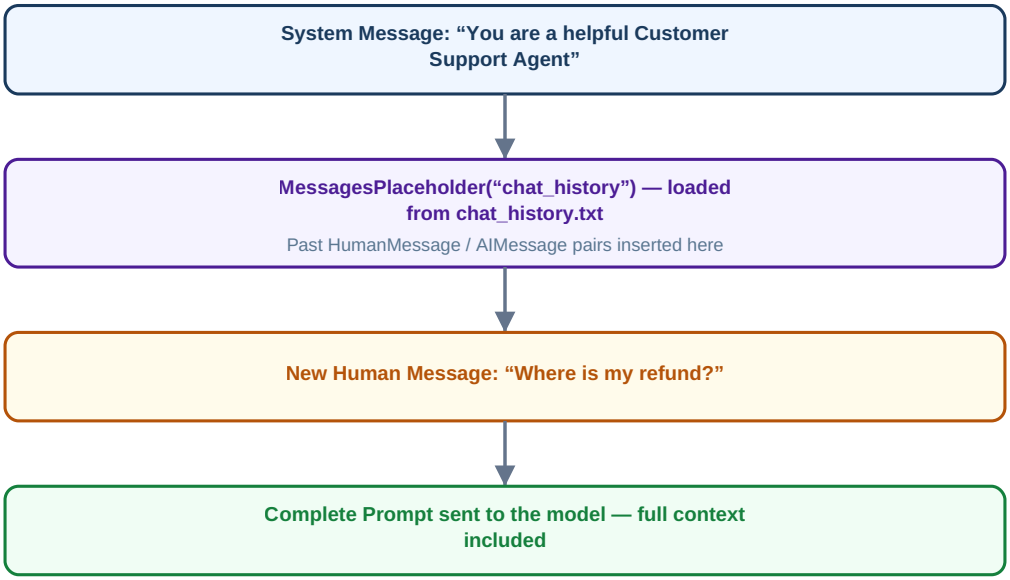

Figure 4.3 — MessagesPlaceholder sits between the system message and the newest human message, injecting the full saved chat history at that position.

Example

```
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

chat_template = ChatPromptTemplate([
    ("system", "You are a helpful Customer Support Agent."),
    MessagesPlaceholder(variable_name="chat_history"),
    ("human", "{query}")
])

chat_history = []
# load the previously saved conversation from disk
with open("chat_history.txt", "r") as file:
    chat_history.extend(file.readlines())

# build the final prompt: system message, then the loaded history, then the new question
prompt = chat_template.invoke({
    "chat_history": chat_history,
    "query": "Where is my refund?"
})

print(prompt)
```

The resulting prompt places the system message first, the entire loaded chat history next, and the customer's new question last — giving the model everything it needs to connect "Where is my refund?" back to order #12345 automatically.

REAL WORLD INSIGHT

In production, chat history is typically persisted in a proper database rather than a flat text file, so it can be reloaded correctly per user session. The core mechanism — loading history, then feeding it into a MessagesPlaceholder — stays the same.

↻ Quick Revision — MessagesPlaceholder

- MessagesPlaceholder inserts a full list of past messages into a ChatPromptTemplate at a fixed position.
- It is typically placed between the system message and the newest human message.
- Its variable_name (e.g. 'chat_history') is the key used when calling .invoke() to supply the saved history.
- This pattern lets a chatbot resume a conversation with full context, even across separate sessions.

K. Key Takeaways

- simply the message sent to an LLM; output quality is highly sensitive to prompt wording.
- the user types the full prompt freely — flexible but inconsistent.
- a fixed template with placeholders filled from constrained user input — reliable and consistent.
- defines a reusable, validated prompt template; beats f-strings via validation, save/load reusability, and chain compatibility.
- (template | model) combine filling a template and calling the model into a single .invoke().
- must be sent in full on every turn, or the model loses conversational context.
- SystemMessage, HumanMessage, and AIMessage label who said what, keeping long conversations coherent.
- the multi-message version of PromptTemplate, built from (role, template_string) tuples.
- dynamically injects a full saved chat history into a ChatPromptTemplate at a fixed position.

R. Revision Sheet

Prompt

The message sent to an LLM; output is highly sensitive to how it's worded.

Static Prompt

User-typed, free-form prompt text with no enforced structure.

Dynamic Prompt

A fixed template whose placeholders are filled from constrained user input.

PromptTemplate

LangChain class for defining, validating, saving, and reusing a single dynamic prompt.

Chain (template | model)

Combines filling a template and invoking the model into one .invoke() call.

SystemMessage / HumanMessage / AIMessage

LangChain's three message types, labeling instructions, user input, and model replies respectively.

ChatPromptTemplate

Builds a list of role-tagged messages with placeholders, built from (role, template_string) tuples.

MessagesPlaceholder

Injects a full stored chat history into a ChatPromptTemplate at a specific position.

I. Interview Questions

Q1. What is a prompt, and why is prompt wording so important when working with LLMs?

Q2. What is the difference between a static prompt and a dynamic prompt, and why are dynamic prompts generally preferred in production applications?

Q3. Why would you use LangChain's PromptTemplate class instead of a plain Python f-string?

Q4. What does validate_template=True do in a PromptTemplate, and why is it useful?

Q5. How does chaining a template and a model together with the pipe operator (template | model) simplify code compared to calling .invoke() twice?

Q6. Why does a chatbot that only sends the latest user message to model.invoke() fail on follow-up questions?

Q7. What are the three message types in LangChain, and what does each represent?

Q8. What problem would arise if a chatbot stored its chat history as plain strings instead of typed message objects?

Q9. What is the difference between PromptTemplate and ChatPromptTemplate?

Q10. What role does MessagesPlaceholder play inside a ChatPromptTemplate, and where is it typically positioned?

F. Further Reading

- LangChain Python Documentation — Prompt Templates
- LangChain Python Documentation — Chat Prompt Templates & MessagesPlaceholder
- LangChain Python Documentation — Messages (SystemMessage, HumanMessage, AIMessage)
- Hugging Face Model Hub — Qwen/Qwen2.5-7B-Instruct model card
- Streamlit Documentation — Input Widgets (text_input, selectbox, button)